A PROJECT ON SQL

A STUDY ON AMAZON DATABASE

CREATING A DATABASE NAMED 'AMAZON_DB'

Access Your Database Server:

Log in to your DBMS (e.g., MySQL, PostgreSQL) via a terminal, command-line interface, or GUI tool (e.g., MySQL Workbench, pgAdmin).

Run the Create Command:

Use the `CREATE DATABASE` command to create a new database.

Example for MySQL/PostgreSQL:

```
CREATE DATABASE Amazon_db;
```

Switch to the new database

+ Use the database to start creating tables and working with data.

  Example: USE Amazon_db;

Verify the Database:

+ List all databases to confirm creation

  Example: SHOW databases;

CREATING TABLES INTO DATABASES

- **STEPS TO BE FOLLOWED WHILE CREATING TABLES INTO DATABASES**
- **Common Data Types:**
- **INT:** Integer numbers.
- **VARCHAR(size):** Variable-length strings.
- **DATE:** For storing dates.
- **DECIMAL(m, n):** Fixed-point numbers.
- **TEXT:** Long text data.
- **Common Constraints:**
- **PRIMARY KEY:** Unique identifier for each row.
- **NOT NULL:** Ensures the column cannot have `NULL` values.
- **UNIQUE:** Ensures all values in the column are unique.
- **FOREIGN KEY:** Links the column to a column in another table.

Steps to Create a Table in a Database:

+ **Choose a Database:** Before creating tables, ensure you're using the correct database:

  Example: USE Amazon_db;

Define the Table Structure: Use the `CREATE TABLE` statement to define the table, its columns, and their data types.

Example syntax:

```
CREATE TABLE table_name (
    column1_name data_type constraints,
    column2_name data_type constraints,
    ...
);
```

CREATE A TABLE 'USERS'

```sql
-- 2) creating a table
-- create a table named 'Users';
CREATE TABLE Users (
user_id int auto_increment primary key,
name varchar(100) not null,
email varchar(150) unique not null,
registered_date date not null,
membership ENUM('basic', 'prime') DEFAULT 'basic'
);
```

CREATE A TABLE 'Products'

```sql
-- create a table named 'Products';
CREATE TABLE Products (
product_id int auto_increment primary key,
name varchar(200) not null,
price decimal(10,2)  not null,
category varchar(100) not null,
stock int not null
);
```

Create a table 'Orders'

```sql
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

Create a table 'Orderdetails'

```sql
-- create a table named 'Orderdetails ';
CREATE TABLE Orderdetails (
    order_details_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    product_id INT,
    FOREIGN KEY (product_id) REFERENCES Products(product_id),
    quantity INT NOT NULL
);
```

Inserting columns into tables

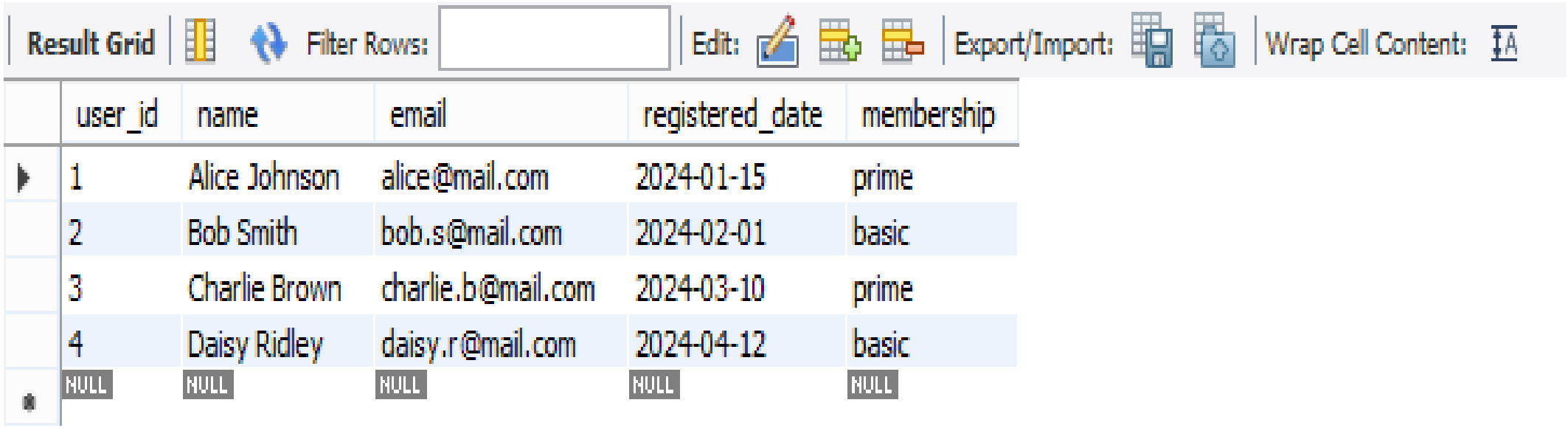
- To insert data into tables, use the `INSERT INTO` statement. Here's how you can do it step by step:
- SYNTAX:
- INSERT INTO table_name (column1, column2, column3, ...)
- VALUES (value1, value2, value3, ...);

INSERT INTO TABLES 'USERS'

```sql
-- 3) Inserting columns into tables
-- inserting into table 'Users'
INSERT INTO Users (name ,email ,registered_date ,membership )
VALUES
('Alice Johnson','alice@mail.com','2024-01-15','Prime'),
('Bob Smith','bob.s@mail.com','2024-02-01','Basic'),
('Charlie Brown','charlie.b@mail.com','2024-03-10','Prime'),
('Daisy Ridley','daisy.r@mail.com','2024-04-12','Basic');
```

OUTPUT:

+ SELECT * FROM 'Users'



user_id	name	email	registered_date	membership
1	Alice Johnson	alice@mail.com	2024-01-15	prime
2	Bob Smith	bob.s@mail.com	2024-02-01	basic
3	Charlie Brown	charlie.b@mail.com	2024-03-10	prime
4	Daisy Ridley	daisy.r@mail.com	2024-04-12	basic
NULL	NULL	NULL	NULL	NULL

INSERT INTO TABLE 'PRODUCTS'

```
-- INSERTING INTO TABLE 'pRODUCts'
INSERT INTO Products (name ,price ,category ,stock)
VALUES
('Echo Dot',49.99,'Electronics',120),
('Kindle Paperwhite',129.99,'Books',50),
('Fire Stick',39.99,'Electronics',80),
('Yoga Mat',19.99,'Fitness',200),
('Wireless Mouse',24.99,'Electronics',150);
```

OUTPUT:

select * from Products;

product_id	name	price	category	stock
1	Echo Dot	49.99	Electronics	120
2	Kindle Paperwhite	129.99	Books	50
3	Fire Stick	39.99	Electronics	80
4	Yoga Mat	19.99	Fitness	200
5	Wireless Mouse	24.99	Electronics	150
NULL	NULL	NULL	NULL	NULL

INSERT INTO TABLE 'ORDERS'

```
-- INSERTING INTO TABLE 'Orders'
INSERT INTO Orders (user_id ,order_date,total_amount)
VALUES
(1,'2024-05-01',79.98),
(2,'2024-05-03',129.99),
(1,'2024-05-04',49.99),
(3,'2024-05-05',24.99);
```

OUTPUT:

order_id	order_date	total_amount	user_id
1	2024-05-01	79.98	1
2	2024-05-03	129.99	2
3	2024-05-04	49.99	1
4	2024-05-05	24.99	3
NULL	NULL	NULL	NULL

INSERT INTO TABLE 'ORDERDETAILS'

```sql
-- INSERTING INTO TABLE 'Orderdetails'
INSERT INTO Orderdetails (order_id,product_id,quantity)
VALUES
(1,1,2),
(2,2,1),
(3,1,1),
(4,5,1);
```

OUTPUT:

order_details_id	order_id	product_id	quantity
1	1	1	2
2	2	2	1
3	3	1	1
4	4	5	1
NULL	NULL	NULL	NULL

LIST ALL CUSTOMERS WHO HAVE MADE OF MORE THAN $80

```sql
-- LIST ALL CUSTOMERS WHO HAVE MADE OF MORE THAN $80
SELECT
    users.user_id,
    users.name,
    users.email,
    SUM(orders.total_amount) AS total_spent
FROM
    Users users
JOIN
    Orders orders
ON
    users.user_id = orders.user_id
GROUP BY
    users.user_id, users.name, users.email
HAVING
    SUM(orders.total_amount) > 80
LIMIT 0, 1000;
```

OUTPUT: The persons who have spent more than $80.

	user_id	name	email	total_spent
▶	1	Alice Johnson	alice@mail.com	129.97
	2	Bob Smith	bob.s@mail.com	129.99

+ **Explanation:**
+ **`SELECT u.user_id, u.username, u.email:`**
  - Retrieves the customer information from the `Users` table.
+ **`SUM(o.total_amount) AS total_spent:`**
  - Calculates the total amount spent by each customer.
+ **`JOIN Users u ON u.user_id = o.user_id:`**
  - Joins the `Users` and `Orders` tables using the `user_id` foreign key.
+ **`GROUP BY u.user_id, u.username, u.email:`**
  - Groups the results by each customer so that the total spent is calculated per customer.
+ **`HAVING SUM(o.total_amount) > 80:`**
  - Filters the results to include only customers who spent more than $80.

Retrieve all orders placed in the last 280 days along with the customer name and email.

```sql
-- B) Retrieve all orders placed in thhe last 280 days along with the customer name and email

SELECT
    users.name,
    users.email,
    orders.order_id,
    orders.order_date,
    orders.total_amount
FROM
    Orders orders
JOIN
    Users users
ON
    orders.user_id = users.user_id
WHERE
    orders.order_date >= DATE_SUB(CURDATE(), INTERVAL 280 DAY)
ORDER BY
    orders.order_date DESC;
```

Output:



name	email	order_id	order_date	total_amount
Charlie Brown	charlie.b@mail.com	4	2024-05-05	24.99
Alice Johnson	alice@mail.com	3	2024-05-04	49.99
Bob Smith	bob.s@mail.com	2	2024-05-03	129.99
Alice Johnson	alice@mail.com	1	2024-05-01	79.98

+ **Explanation:**
+ **`SELECT u.name, u.email:`**
  - Retrieves the customer's name and email from the `Users` table.
+ **`o.order_id, o.order_date, o.total_amount:`**
  - Retrieves the order details from the `Orders` table.
+ **`JOIN Users u ON o.user_id = u.user_id:`**
  - Joins the `Orders` table with the `Users` table using the `user_id` foreign key.
+ **`WHERE o.order_date >= DATE_SUB(CURDATE(), INTERVAL 280 DAY):`**
  - Filters orders placed in the last 280 days from the current date.
+ **`ORDER BY o.order_date DESC:`**
  - Orders the results by `order_date` in descending order (latest orders first).

Find the average product price for each category

```sql
-- c) Find the average product price for each category
SELECT
    category,
    AVG(price) AS average_price
FROM
    Products
GROUP BY
    category
ORDER BY
    average_price DESC;
```

Output:

Result Grid | Filter Rows:

category	average_price
Books	129.990000
Electronics	38.323333
Fitness	19.990000

+ **Explanation:**
+ **`SELECT category, AVG(price) AS average_price:`**

  •Retrieves the product category and calculates the average price using the `AVG` function.

+ **`FROM Products:`**

  •Specifies the `Products` table as the source of the data.

+ **`GROUP BY category:`**

  •Groups the results by each product category to calculate the average price for each group.

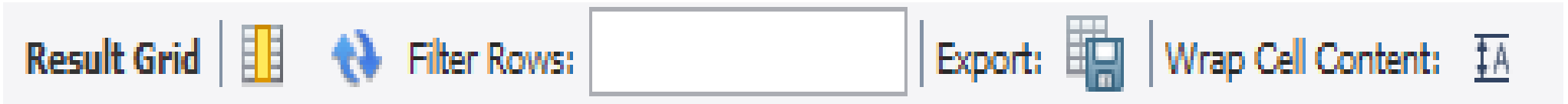
+ **`ORDER BY average_price DESC:`**

  •Sorts the results in descending order of the average price.

List all customers who have purchased a product from the category Electronics.

```sql
-- d) List all customers who have purchased a product from the category Electronics.
SELECT DISTINCT
    users.name,
    users.email
FROM
    Users users
JOIN
    Orders orders ON users.user_id = orders.user_id
JOIN
    OrderDetails orderdetails  ON orders.order_id = orderdetails.order_id
JOIN
    Products products ON orderdetails.product_id = products.product_id
WHERE
    products.category = 'Electronics'
ORDER BY
    users.name;
```

Output:



	name	email
▶	Alice Johnson	alice@mail.com
	Charlie Brown	charlie.b@mail.com

+ **Explanation:**
+ **`SELECT DISTINCT u.name, u.email:`**
  - Retrieves unique customer names and emails to avoid duplicates.
+ **`JOIN Orders o ON u.user_id = o.user_id:`**
  - Joins the `Users` table with the `Orders` table to get customer orders.
+ **`JOIN OrderDetails od ON o.order_id = od.order_id:`**
  - Joins the `Orders` table with the `OrderDetails` table to get product details for each order.
+ **`JOIN Products p ON od.product_id = p.product_id:`**
  - Joins the `OrderDetails` table with the `Products` table to filter by product category.
+ **`WHERE p.category = 'Electronics':`**
  - Filters the results to include only products in the "Electronics" category.
+ **`ORDER BY u.name:`**
  - Sorts the results alphabetically by customer name.

Find the total no of products and the total revenue generated for each product.

```sql
-- e) Find the total no of products and the total revenue generated for each product.

SELECT
    products.name,
    SUM(quantity) AS total_quantity_sold,
    SUM(quantity * price) AS total_revenue
FROM
    OrderDetails OrderDetails
JOIN
    Products products
ON
    OrderDetails.product_id = products.product_id
GROUP BY
    products.product_id, products.name
ORDER BY
    total_revenue DESC;
```

Output

name	total_quantity_sold	total_revenue
Echo Dot	3	149.97
Kindle Paperwhite	1	129.99
Wireless Mouse	1	24.99

+ **Explanation:**
+ **`SELECT product.name:`**

  •Retrieves the product name from the `Products` table.
+ **`SUM(quantity) AS total_quantity_sold:`**

  •Calculates the total number of units sold for each product.
+ **`SUM(quantity * price) AS total_revenue:`**

  •Calculates the total revenue generated by multiplying the quantity sold by the product price.
+ **`JOIN Products p ON od.product_id = p.product_id:`**

  •Joins the `OrderDetails` table with the `Products` table based on `product_id`.
+ **`GROUP BY p.product_id, p.product_name:`**

  •Groups the results by each product to calculate totals for each product.
+ **`ORDER BY total_revenue DESC:`**

  •Sorts the results in descending order of revenue.

Update the price of all products in the books category, increasing it by 10%

```sql
-- f) Update the price of all products in the books category, increasing it by 10%
SET SQL_SAFE_UPDATES = 0;


UPDATE Products
SET price = price * 1.10
WHERE category = 'Books';
SET SQL_SAFE_UPDATES = 1;
```

Output:

	product_id	name	price	category	stock
	1	Echo Dot	49.99	Electronics	120
..✎	2	Kindle Paperwhite	142.99	Books	50
	3	Fire Stick	39.99	Electronics	80
	4	Yoga Mat	19.99	Fitness	200
	5	Wireless Mouse	24.99	Electronics	150

+ **Explanation:**
+ **`UPDATE Products:`**

  •Specifies the `Products` table where the prices will be updated.
+ **`SET price = price * 1.10:`**

  •Multiplies the existing price by `1.10`, effectively increasing the price by 10%.
+ **`WHERE category = 'Books':`**

  •Filters the products to only those in the "Books" category.

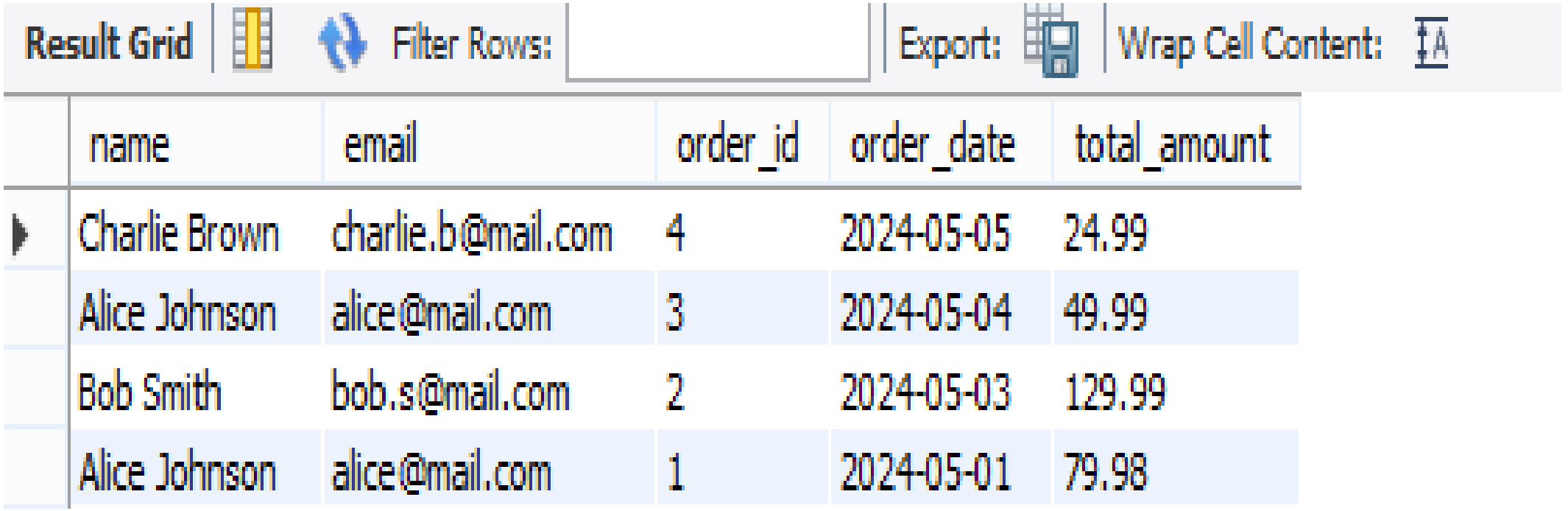
Remove all orders that were placed before 2020

```sql
-- g) Remove all orders that were placed before 2020
SET SQL_SAFE_UPDATES = 0;

DELETE FROM Orders
WHERE order_date < '2020-01-01';

SET SQL_SAFE_UPDATES = 1;
```

OUTPUT:



name	email	order_id	order_date	total_amount
Charlie Brown	charlie.b@mail.com	4	2024-05-05	24.99
Alice Johnson	alice@mail.com	3	2024-05-04	49.99
Bob Smith	bob.s@mail.com	2	2024-05-03	129.99
Alice Johnson	alice@mail.com	1	2024-05-01	79.98

+ **Explanation:**
+ **`DELETE FROM Orders:`**

  •Specifies that you want to delete rows from the `Orders` table.

+ **`WHERE order_date < '2020-01-01':`**

  •Filters the rows to delete only those with an `order_date` earlier than January 1, 2020.

Write a query to fetch the order details, including customer name,product name, and quantity , for orders placed on 2024-05-01

```sql
-- H) Write a query to fetch the order details, including customer name,product name,
SELECT
    users.name AS customer_name,
    products.name,
    orderdetails.quantity
FROM
    Orders orders
JOIN
    Users users ON orders.user_id = users.user_id
JOIN
    OrderDetails orderdetails ON orders.order_id = orderdetails.order_id
JOIN
    Products products ON orderdetails.product_id = products.product_id
WHERE
    orders.order_date = '2024-05-01';
```

Output:

	customer_name	name	quantity
▶	Alice Johnson	Echo Dot	2

+ **Explanation:**
+ **`SELECT u.first_name AS customer_name:`**

  •Retrieves the customer's first name. Adjust to include last name if needed (e.g., `CONCAT(u.first_name, ' ', u.last_name)`).
+ **`p.product_name:`**

  •Retrieves the name of the product from the `Products` table.
+ **`od.quantity:`**

  •Retrieves the quantity of the product ordered.
+ **`FROM Orders o:`**

  •Specifies the `Orders` table as the base.
+ **`JOIN Users u ON o.user_id = u.user_id:`**

  •Joins the `Orders` table with the `Users` table to fetch customer information.
+ **`JOIN OrderDetails od ON o.order_id = od.order_id:`**

  •Joins the `Orders` table with the `OrderDetails` table to get details about the products in each order.
+ **`JOIN Products p ON od.product_id = p.product_id:`**

  •Joins the `OrderDetails` table with the `Products` table to get product names.
+ **`WHERE o.order_date = '2024-05-01':`**

  •Filters for orders placed on the specific date.

Fetch all customers and the total number of orders they have placed

```sql
-- i) Fetch all customers and the total number of orders they have placed

SELECT
    users.user_id,
    CONCAT(users.name) AS customer_name,
    COUNT(orders.order_id) AS total_orders
FROM
    Users users
LEFT JOIN
    Orders orders ON users.user_id = orders.user_id
GROUP BY
    users.user_id, users.name
ORDER BY
    total_orders DESC;
```

Output:

user_id	customer_name	total_orders
1	Alice Johnson	2
2	Bob Smith	1
3	Charlie Brown	1
4	Daisy Ridley	0

+ **Explanation:**

1. SELECT u.user_id, CONCAT(u.name) AS customer_name:

1. Retrieves the customer's unique ID and full name. Adjust based on the column names in your schema.

2. COUNT(o.order_id) AS total_orders:

1. Counts the total number of orders placed by each customer.

3. FROM Users u:

1. Specifies the Users table as the base table.

4. LEFT JOIN Orders o ON u.user_id = o.user_id:

1. Joins the Orders table to the Users table to include all users, even those who haven't placed any orders.

5. GROUP BY u.user_id, u.name:

1. Groups the results by customer to calculate the total orders for each.

6. ORDER BY total_orders DESC:

1. Orders the results by the total number of orders in descending order.

Conclusion

- The database is well-structured, enabling efficient tracking of customers, products, orders, and categories. It provides valuable insights into:
- **Customer Behavior**: Identifying purchasing patterns and loyal customers.
- **Revenue Analysis**: Breaking down total revenue by product categories to find top-performing ones.
- **Sales Trends**: Highlighting best-selling products and high-demand items.
- **Scalability**: Supports easy expansion for new data without disrupting the structure.